

BACKEND TECHNICAL SPECIFICATION

Sentiora

| Never lose what you learned.

Document: Backend Technical Specification

Version: 1.0

Status: Draft for Engineering Review

Scope: Sentiora Memory & Sentiora Intelligence (MVP)

Audience: Backend Engineering Team (4-person team)

Author: Principal Backend Architecture Function

Related Documents: Sentiora PRD v2.1 (FINAL), SRS, UI/UX Specification, Database Schema

Table of Contents

1. Introduction	3
2. Backend Goals	4
3. Overall Backend Architecture	5
4. Project Structure	7
5. Authentication & Authorization	9
6. API Architecture	11
7. Data Processing Pipeline	13
8. Content Processing	16
9. Embedding Pipeline	18
10. Semantic Search & RAG	19
11. Background Jobs (Celery)	21
12. Storage Architecture	23
13. Error Handling & Logging	24
14. Security & Privacy	26
15. Deployment	28
16. Cross-Cutting Concerns	29
17. Future Module Extension (Footprint & Shield)	30
18. Development Roadmap	32

1. Introduction

1.1 Purpose

This document is the Backend Technical Specification (BTS) for Sentiora. It defines **how** the backend system is designed, structured, and implemented — the concrete engineering blueprint that a backend engineer uses to write code. It does not repeat product rationale, UI behavior, or column-level schema definitions; those live in the PRD, UI/UX Specification, and Database Schema documents respectively. Where this document references data structures, it does so only to the extent required to explain backend behavior.

1.2 Scope

This specification covers the complete backend implementation of the two MVP modules — **Sentiora Memory** and **Sentiora Intelligence** — including authentication, API architecture, the content processing pipeline, the embedding pipeline, semantic search and RAG, background job processing, storage architecture, error handling, security, and deployment. For **Sentiora Footprint** and **Sentiora Shield**, this document defines only backend extension points (Section 17) so that the MVP architecture does not need to be reworked when those modules are built.

Out of scope for this document: frontend implementation, UI components, detailed database schema/DDL, endpoint-by-endpoint API reference (covered in a separate API Specification), mobile applications, analytics dashboards, billing, Kubernetes, microservices decomposition, and multi-region infrastructure. The MVP backend is a single deployable FastAPI service with a layered internal architecture — not a microservices system.

1.3 Audience

This document is written for backend engineers joining the Sentiora project, including engineers with no prior context on the product. Familiarity with Python, REST APIs, relational databases, and asynchronous task queues is assumed; familiarity with Sentiora itself is not.

1.4 Referenced Documents

Document	Purpose	Relationship to this Document
Sentiora PRD v2.1 (FINAL)	Product requirements, scope, personas, success criteria	Source of truth for <i>what</i> to build; this document defines <i>how</i>
Software Requirements Specification (SRS)	Functional/non-functional requirement traceability	Backend components map to SRS requirement IDs where relevant
UI/UX Specification	Extension UI flows, consent screens, capture controls	Defines the client contract the API layer must satisfy
Database Schema	Table definitions, columns, constraints, indexes	This document describes data <i>flow</i> and <i>access patterns</i> , not column definitions

2. Backend Goals

The backend exists to reliably turn permissioned user activity into structured, searchable memory, and to answer questions over that memory safely and quickly. Every architectural decision in this document is judged against six goals.

Goal	What It Means Here	How It Is Achieved
Modular Architecture	Memory, Intelligence, and future Footprint/Shield modules can be developed, tested, and reasoned about independently	Strict layering (API → Service → Repository), one service module per domain, no cross-module direct DB access
Performance	Capture must never block the user's browsing; search/chat must feel conversational	Synchronous APIs stay thin; heavy work (extraction, embedding, LLM calls) is offloaded to Celery; pgvector indexes tuned for approximate nearest-neighbor search; Redis caching for hot reads
Security	User memory is sensitive personal data by definition	JWT-based auth on every protected route, strict per-user data isolation at the repository layer, secrets never in code, input validation at the schema boundary
Privacy	Sentiora's trust proposition depends on the backend enforcing consent, not just the UI presenting it	Permission validation is a first-class pipeline stage (Section 7.2) enforced server-side, never trusted from the client alone; never-capture policy enforced before any persistence
Scalability	Support growth in users and in memory volume per user without redesign	Stateless API instances behind a load balancer, horizontally scalable Celery workers, vector index strategy that scales with corpus size
Maintainability	A 4-person team must be able to safely extend the system over multiple phases	Repository pattern isolates persistence details, dependency injection for testability, consistent error/response contracts, documented extension points for Footprint/Shield

Design principle: the backend optimizes for *correct, permissioned memory* over *completeness of capture*. Every pipeline and API decision defaults to excluding uncertain or sensitive data rather than including it, consistent with the "Meaningful Capture > Comprehensive Logging" principle established in the PRD.

3. Overall Backend Architecture

Sentiora's backend is a single layered FastAPI service backed by PostgreSQL (with pgvector), Redis, S3-compatible object storage, and OpenAI's embedding and LLM APIs. Heavy or latency-sensitive work is delegated to Celery workers so the synchronous API path stays fast. The diagram below shows the full request and data path from the Chrome Extension down to the LLM.

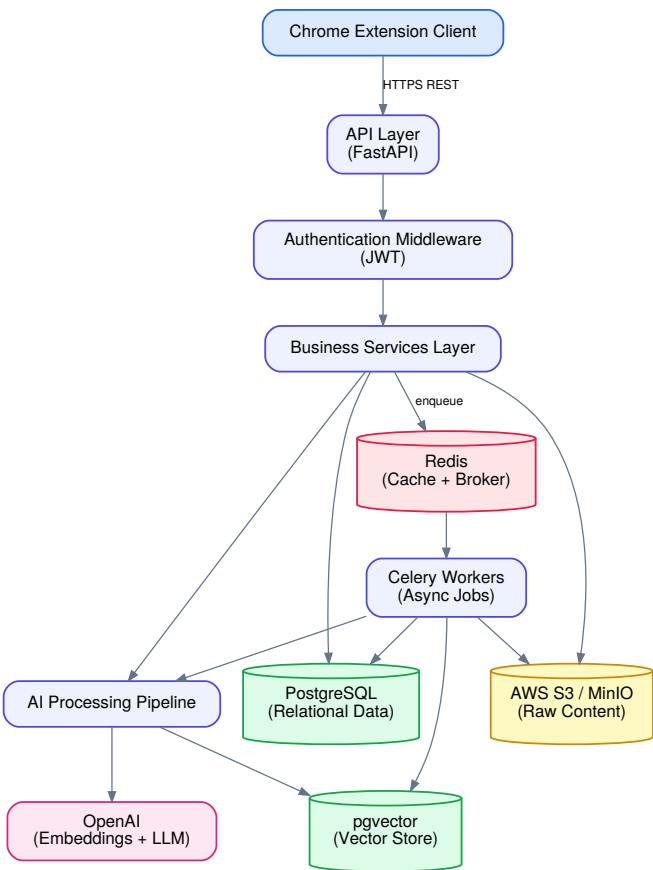


Figure 3.1 — End-to-end backend architecture, from client to LLM

3.1 Layer Responsibilities

Layer	Responsibility	Key Technology
Chrome Extension Client	Captures user-approved content, renders consent UI, calls backend APIs over HTTPS	Not covered by this document
API Layer	Request routing, schema validation, response shaping	FastAPI, Pydantic
Authentication	Identity verification, token issuance/validation	JWT (access + refresh), passlib/bcrypt
Business Services	Domain logic: capture rules, permission checks, search orchestration, chat orchestration	Python service classes, dependency-injected
AI Processing Pipeline	Extraction, cleaning, chunking, embedding generation, RAG orchestration	Custom pipeline modules, OpenAI SDK
Database	Durable relational storage of users, memories, sources, jobs	PostgreSQL, SQLAlchemy ORM
Vector Database	Similarity search over memory chunk embeddings	pgvector extension inside PostgreSQL
Storage	Raw captured content, large payloads, PDFs	AWS S3 (MinIO in development)
LLM	Embedding generation, chat/summarization responses	OpenAI text-embedding-3-small, GPT-4.1 / GPT-5

3.2 Why a Layered Monolith, Not Microservices

A 4-person team building an MVP benefits from a single deployable unit: one codebase, one CI pipeline, one set of migrations, no distributed-transaction complexity. Modularity is achieved through internal layering and clear service boundaries (Section 4), not through network-separated services. This keeps the system simple to operate while still allowing Memory, Intelligence, and future modules to be split into separate services later if scale demands it — the repository and service boundaries are designed to make that split mechanical rather than a rewrite.

3.3 Synchronous vs. Asynchronous Paths

Path	Examples
------	----------

Execution Mode		
Fast, user-facing	Login, fetch memory timeline, run a search query, send a chat message	Synchronous request/response through FastAPI
Slow, background	Content extraction, embedding generation, re-indexing, retries	Asynchronous via Celery, status polled or pushed to client

The rule of thumb: if an operation's latency depends on an external system (LLM call, large PDF parse, YouTube transcript fetch) or is not required for the user to continue their immediate action, it is offloaded to a Celery task rather than executed inline in the request handler.

4. Project Structure

The folder structure enforces the layered architecture described in Section 3. Each top-level folder maps to exactly one architectural concern; a new engineer should be able to guess where a given piece of logic lives from this table alone.

```
backend/
├── api/ # route definitions only — no business logic
│   ├── v1/
│   │   ├── auth_routes.py
│   │   ├── memory_routes.py
│   │   ├── search_routes.py
│   │   ├── chat_routes.py
│   │   ├── source_routes.py
│   │   └── health_routes.py
│   └── deps.py # FastAPI dependency-injection providers
├── auth/ # JWT issuance, password hashing, token validation
│   ├── jwt_handler.py
│   ├── password.py
│   └── permissions.py
├── core/ # app-wide config, settings, startup/shutdown
│   ├── config.py
│   ├── logging.py
│   └── security.py
├── models/ # SQLAlchemy ORM models (mirrors DB schema, not redefined here)
│   ├── user.py
│   ├── memory.py
│   ├── source.py
│   ├── chunk.py
│   └── job.py
├── schemas/ # Pydantic request/response contracts
│   ├── auth_schemas.py
│   ├── memory_schemas.py
│   ├── search_schemas.py
│   └── chat_schemas.py
├── repositories/ # all DB queries live here — services never query directly
│   ├── user_repository.py
│   ├── memory_repository.py
│   ├── chunk_repository.py
│   └── job_repository.py
├── services/ # business logic, orchestration, domain rules
│   ├── auth_service.py
│   ├── capture_service.py
│   ├── permission_service.py
│   ├── search_service.py
│   ├── chat_service.py
│   └── citation_service.py
├── workers/ # Celery app, task definitions, queue routing
│   ├── celery_app.py
│   ├── capture_tasks.py
│   ├── embedding_tasks.py
│   └── maintenance_tasks.py
├── embeddings/ # embedding client, batching, retry logic
│   ├── embedding_client.py
│   └── batcher.py
├── rag/ # retrieval, ranking, prompt assembly, citation mapping
│   ├── retriever.py
│   ├── ranker.py
│   ├── prompt_builder.py
│   └── llm_client.py
├── processing/ # content extraction per source type
│   ├── web_extractor.py
│   ├── pdf_extractor.py
│   ├── youtube_extractor.py
│   ├── github_extractor.py
│   ├── cleaner.py
│   ├── normalizer.py
│   └── chunker.py
├── middleware/ # request ID, logging, error handling, auth middleware
│   ├── request_id.py
│   ├── error_handler.py
│   └── auth_middleware.py
├── utils/ # pure helper functions, no side effects
└── tests/ # unit +integration tests, mirrors source structure
```

```
└─ alembic/ # DB migrations
└─ main.py # FastAPI app instantiation, router registration
└─ Dockerfile
└─ docker-compose.yml
└─ requirements.txt
```

4.1 Folder-by-Folder Responsibilities

Folder	Owns	Must Not Contain
api/	Route declarations, request parsing, calling into services	Business rules, direct DB or ORM queries
auth/	Token lifecycle, password hashing primitives	User-facing business rules (e.g., what a free-tier user can capture)
models/	ORM class definitions matching the Database Schema document	Query logic — that belongs in repositories/
schemas/	Input/output validation contracts	Database field definitions (these can diverge intentionally from ORM models)
repositories/	All SQL/ORM query construction, single point of DB access per entity	Business decisions (e.g., "should this be captured?")
services/	Orchestration of repositories, processing, embeddings, and rules	Raw SQL, HTTP route decorators
workers/	Celery task registration and queue routing	New business logic — tasks call into services/
embeddings/	OpenAI embedding calls, batching, backoff/retry	Vector storage logic — that belongs in repositories/chunk_repository.py
rag/	Retrieval, ranking, prompt construction, LLM invocation for chat	Persistence logic
processing/	Source-specific extraction and content normalization	Embedding generation or DB writes
middleware/	Cross-cutting request/response concerns	Domain-specific logic

5. Authentication & Authorization

Sentiora uses stateless JWT-based authentication with short-lived access tokens and long-lived, rotating refresh tokens. Authorization is enforced per-request by binding every downstream repository query to the authenticated `user_id` extracted from the token — never from a client-supplied parameter.

5.1 Registration

1. Client submits email + password to `POST /auth/register`.
2. `AuthService` validates email uniqueness via `UserRepository`.
3. Password is hashed with bcrypt (cost factor 12) — plaintext password is never logged or persisted.
4. A new user record is created; an `onboarding_state` flag is initialized for extension-side consent flows.

5.2 Login & Token Issuance

Token	Lifetime	Storage (Client)	Purpose
Access Token	15 minutes	Memory / short-lived extension storage	Sent as <code>Authorization: Bearer</code> on every protected request
Refresh Token	14 days, rotated on use	Secure extension storage	Exchanged for a new access/refresh pair via <code>/auth/refresh</code>

Refresh tokens are single-use: each refresh call invalidates the presented token and issues a new one. The previous token's identifier is recorded in Redis with a TTL matching its original expiry, so a reused (stolen) refresh token is detected and rejected, and the associated session can be revoked.

5.3 Password Hashing

Passwords are hashed with bcrypt via `passlib`. Sentiora never stores or transmits raw passwords beyond the initial HTTPS request, and password hashes are excluded from every serialization schema by default (opt-in only, never opt-out).

5.4 Protected APIs & Authorization Middleware

Every route except `/auth/register`, `/auth/login`, `/auth/refresh`, and `/health` requires a valid access token. The authorization middleware:

1. Extracts the bearer token from the `Authorization` header.
2. Validates signature and expiry using the JWT secret from Secrets Manager / environment configuration.
3. Decodes `user_id` and attaches it to the request context via FastAPI's dependency injection (`get_current_user`).
4. Rejects with `401 Unauthorized` on any failure — expired, malformed, or missing token.

Data isolation rule: no repository method may accept a bare `memory_id` or `source_id` without also filtering by the `user_id` from the authenticated context. This is enforced at the repository layer, not just the service layer, so a bug in one service cannot leak cross-user data.

5.5 Authentication Sequence

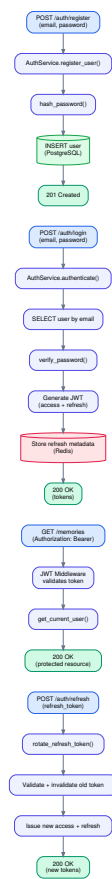


Figure 5.1 — Registration, login, protected access, and refresh flows

6. API Architecture

6.1 Request Lifecycle

Every request passes through the same layered pipeline regardless of endpoint. This consistency is what lets a new engineer safely add an endpoint without re-deriving the architecture.

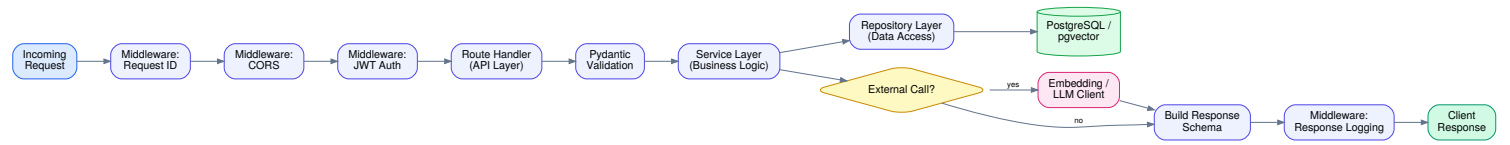


Figure 6.1 — Request lifecycle from client to response

6.2 Routing

Routes are versioned under `/api/v1/` and grouped by domain (`auth`, `memory`, `search`, `chat`, `source`). Route handlers are intentionally thin: parse input via a Pydantic schema, call exactly one service method, return the service's result mapped to a response schema.

```
# api/v1/memory_routes.py
@router.post("/memories", response_model=MemoryResponse, status_code=201)
async def create_memory(
    payload: MemoryCreateRequest,
    user: User = Depends(get_current_user),
    memory_service: MemoryService = Depends(get_memory_service),
):
    memory = await memory_service.create_memory(user_id=user.id, payload=payload)
    return MemoryResponse.from_orm(memory)
```

6.3 Validation

All request bodies are validated by Pydantic schemas before reaching a service. Validation failures short-circuit at the API layer and never reach business logic, guaranteeing services can assume well-formed input types (though not necessarily valid business state — that is the service's job to check).

6.4 Business Layer vs. Repository Layer

Concern	Business Layer (services/)	Repository Layer (repositories/)
Question answered	"Is this action allowed and what should happen?"	"How is this data read or written?"
Example	Reject capture of a banking page before it reaches storage	Insert a validated memory row scoped to <code>user_id</code>
Depends on	Repositories, processing modules, embedding/RAG modules	SQLAlchemy session, ORM models
Testability	Unit-tested with repositories mocked	Integration-tested against a test database

6.5 Response Structure

All successful responses follow a consistent envelope so the extension client can handle them uniformly:

```
{
  "data": { ... },
  "meta": {
    "request_id": "req_9f8a3c",
    "timestamp": "2026-07-29T10:15:00Z"
  }
}
```

6.6 Error Responses

```
{
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "This source type is not enabled for capture.",
    "request_id": "req_9f8a3c"
  }
}
```

Full error taxonomy is defined in Section 13.

6.7 Middleware Stack (applied in order)

1. **Request ID** — assigns a unique ID for tracing across logs and Celery tasks.
2. **CORS** — restricts allowed origins to the Chrome Extension's origin.
3. **Authentication** — validates JWT and attaches user context (skipped for public routes).
4. **Error handling** — catches unhandled exceptions and converts them to the standard error envelope.
5. **Logging** — structured request/response logging (Section 13.6).

6.8 Dependency Injection

FastAPI's `Depends()` system is used consistently to provide: the current authenticated user, a database session scoped to the request, and service instances constructed with their repository dependencies. This makes every service and route independently testable by substituting fakes for these dependencies in tests.

7. Data Processing Pipeline

This pipeline is the backbone of Sentiora Memory. It converts a raw capture event into a searchable memory record while enforcing consent and the never-capture policy at every relevant stage. Stages 1-2 run synchronously (fast, must reject before any storage occurs); stages 3 onward run asynchronously via Celery so capture never blocks the user's browsing.

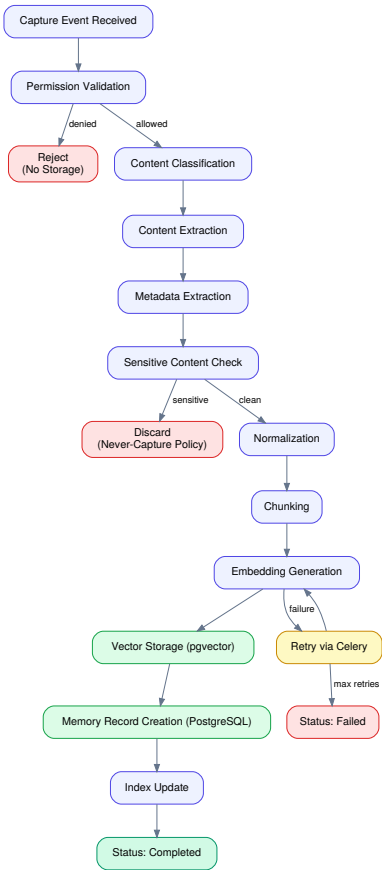


Figure 7.1 — Full capture-to-memory processing pipeline

7.1 Stage Responsibilities

Stage	Runs	Responsibility	Failure Behavior
1. Capture	Sync	Receives the capture payload (URL, page content snapshot, dwell-time signal, source type) from the extension	Malformed payload → <code>422</code> , nothing persisted
2. Permission Validation	Sync	Confirms the source type is enabled in the user's per-source consent settings for this capture	Denied → <code>403</code> , no data written, event discarded
3. Content Classification	Async	Determines source type (web page, PDF, YouTube, GitHub, manual note, AI conversation) to route to the correct extractor	Unknown type → job marked <code>failed</code> , logged for review
4. Content Extraction	Async	Runs the source-specific extractor (Section 8) to obtain raw text content	Extraction error → retried up to 3 times, then <code>failed</code>
5. Metadata Extraction	Async	Pulls title, author, publish date, URL, timestamps, and engagement signals (dwell time)	Partial metadata allowed; required fields missing → <code>failed</code>
6. Sensitive Content Check	Async	Applies the never-capture policy (passwords, payment, banking, health, incognito pages) before any further processing	Match → content discarded permanently, no memory created, audit log entry written
7. Normalization	Async	Strips boilerplate (navigation, ads), normalizes whitespace/encoding, converts to canonical UTF-8 text	N/A — always proceeds with best-effort output
8. Chunking	Async	Splits normalized content into overlapping chunks sized for the embedding model's context window	N/A
9. Embedding Generation	Async	Calls OpenAI text-embedding-3-small per chunk (batched, Section 9)	Retried with exponential backoff; exhausted retries → job <code>failed</code> , memory kept in <code>pending_embedding</code> state
10. Vector Storage	Async	Persists chunk embeddings into pgvector-backed table, linked to the parent memory	DB error → transaction rolled back, retried
11. Memory Creation	Async	Writes the final memory record (metadata + reference to chunks) to PostgreSQL	Wrapped in the same transaction as vector storage where feasible
12. Index Update	Async	Updates any denormalized search indexes (e.g., full-text search columns, tags)	Best-effort; can be re-run by maintenance task

13. Status Tracking	Async	Updates the job status record so the client can poll or be notified of completion	N/A
14. Completion	Async	Marks memory as <code>completed</code> and available for search/chat	N/A

7.2 Permission Validation — Server-Side Enforcement

The extension UI presents consent toggles, but the backend is the actual enforcement point. On every capture request, `PermissionService` checks the user's persisted per-source consent settings before any content leaves the classification stage. This guarantees consent is enforced even if the client is compromised, outdated, or bypassed.

7.3 Never-Capture Policy — Implementation Note

Consistent with the PRD, this document intentionally does not enumerate the specific detection signals used to identify sensitive pages (password fields, payment forms, banking domains, health portals, incognito sessions) — that mechanism is refined at implementation time and treated as a configurable, updatable policy component (`services/permission_service.py`) rather than a hard-coded list. What is fixed: a positive match at Stage 6 results in **immediate, permanent discard** with no partial persistence and no way to recover the discarded content, plus an audit log entry containing only metadata (timestamp, source type, decision) — never the sensitive content itself.

7.4 Idempotency & Job Tracking

Every capture event is assigned a unique `job_id` at Stage 1. All downstream Celery tasks are idempotent with respect to this ID — re-running a task for the same `job_id` must not create duplicate memories or chunks. This is required because Celery's at-least-once delivery guarantee means tasks can be retried or, rarely, executed twice.

8. Content Processing

Each source type has a dedicated extractor module under `processing/`, all implementing a common interface (`extract(raw_input) -> ExtractedContent`) so the pipeline (Section 7) can treat them uniformly after Stage 3 classification.

8.1 Common Extractor Interface

```
# processing/base.py
class ExtractedContent(BaseModel):
    text: str
    title: str | None
    author: str | None
    published_at: datetime | None
    source_url: str | None
    extra_metadata: dict

class BaseExtractor(ABC):
    @abstractmethod
    async def extract(self, raw_input: CaptureInput) -> ExtractedContent: ...
```

8.2 Web Pages

Aspect	Implementation
Extraction	HTML snapshot supplied by the extension is parsed server-side; boilerplate (nav, ads, footers) stripped using readability-style content extraction
Metadata	Title, canonical URL, Open Graph tags, dwell-time signal from the extension
Cleaning	HTML-to-text conversion, whitespace normalization, script/style removal
Storage	Cleaned text persisted for chunking; original HTML snapshot optionally archived to S3 for re-processing, not searched directly

8.3 PDFs

Aspect	Implementation
Extraction	Uploaded PDF stored in S3; text extracted server-side (layout-aware extraction to preserve reading order)
Metadata	PDF metadata fields (title, author, creation date) plus page count
Cleaning	De-hyphenation across line breaks, header/footer repetition removal
Storage	Original PDF retained in S3; extracted text flows into normalization/chunking

8.4 YouTube Videos

Aspect	Implementation
Extraction	Transcript fetched via YouTube's transcript API where available; video metadata (title, channel, duration) fetched separately
Metadata	Channel name, publish date, video URL, timestamp-anchored transcript segments
Cleaning	Auto-caption artifact removal, filler-word normalization
Storage	Transcript text chunked with timestamp references retained in chunk metadata for citation purposes

8.5 GitHub Repositories

Aspect	Implementation
Extraction	README and user-selected files/notes fetched via GitHub API for the linked repository
Metadata	Repository name, owner, file path, last commit reference
Cleaning	Markdown-to-text conversion; code blocks preserved as-is (not stripped) since they carry meaning
Storage	Text chunked per-file; file path retained for citation

8.6 Manual Notes & Links

Aspect	Implementation
Extraction	User-typed text taken as-is; no HTML/PDF parsing needed
Metadata	User-supplied title/tags, creation timestamp

Cleaning	Minimal — whitespace normalization only
Storage	Directly enters normalization/chunking; no S3 archival needed

8.7 AI Conversations

Aspect	Implementation
Extraction	Conversation transcript (user + assistant turns) supplied by the extension from the source AI product's page
Metadata	Source AI product, conversation title, turn count, timestamp
Cleaning	Turn-boundary preservation so chunking does not split mid-turn where avoidable
Storage	Chunked with speaker role retained in chunk metadata for citation and RAG context clarity

9. Embedding Pipeline

9.1 Embedding Generation

Each chunk produced in Section 7.1 Stage 8 is embedded using OpenAI's `text-embedding-3-small` model via `embeddings/embedding_client.py`. Embedding calls are made from Celery tasks, never from the synchronous API path.

```
# embeddings/embedding_client.py
class EmbeddingClient:
    async def embed_batch(self, texts: list[str]) -> list[list[float]]:
        response = await self.client.embeddings.create(
            model="text-embedding-3-small",
            input=texts,
        )
        return [item.embedding for item in response.data]
```

9.2 Batch Processing

Chunks are grouped into batches (default: up to 96 chunks or a token-count ceiling, whichever is reached first) before calling the embedding API, reducing API round-trips and cost. Batching is handled by `embeddings/batcher.py`, which also enforces the model's token-per-request limit.

9.3 Vector Storage

Each embedding is stored in a pgvector column alongside a foreign key to its parent memory and chunk index. Storage and the originating memory record are written within the same logical unit of work so a memory is never left referencing chunks that failed to embed.

9.4 Re-indexing

Re-indexing is required when: the embedding model changes, a chunking strategy change is deployed, or a maintenance task detects orphaned/stale vectors. Re-indexing is a background Celery job (`maintenance_tasks.py`) that re-chunks and re-embeds affected memories without user-facing downtime — old vectors remain searchable until replacement vectors are committed.

9.5 Failure Handling

Failure Type	Handling
Transient API error (rate limit, timeout)	Exponential backoff retry, up to 5 attempts
Persistent API error	Job marked <code>failed</code> ; memory remains in <code>pending_embedding</code> and is excluded from search until resolved
Partial batch failure	Failed chunks retried individually rather than failing the whole batch
Token limit exceeded on a chunk	Chunk re-split smaller and re-submitted

9.6 Embedding Lifecycle

State	Meaning
<code>pending</code>	Chunk created, not yet submitted for embedding
<code>processing</code>	Embedding request in flight
<code>embedded</code>	Vector stored, chunk searchable
<code>failed</code>	Retries exhausted; excluded from search, visible to maintenance tooling
<code>stale</code>	Superseded by a re-indexing pass, pending cleanup

10. Semantic Search & RAG

Sentiora Intelligence exposes two related capabilities on the same retrieval core: **Semantic Search** (returns ranked memory chunks) and **RAG Chat** (returns a synthesized natural-language answer with citations). Both flow through the same retrieval pipeline; chat adds prompt assembly and an LLM call on top.

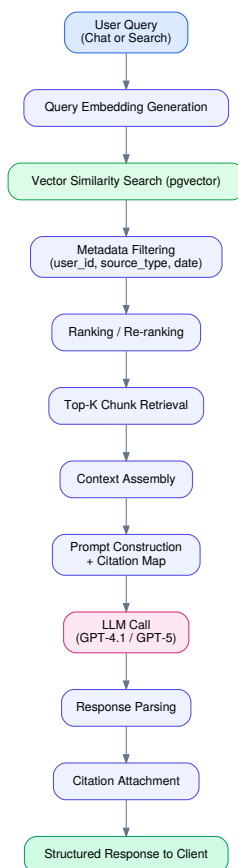


Figure 10.1 — Semantic search and RAG chat flow

10.1 Vector Search

The user's query is embedded using the same model as content chunks (`text-embedding-3-small`), then compared against stored chunk vectors using pgvector's cosine distance operator, scoped to `user_id` via an indexed filter — a user's query never searches another user's vectors, enforced at the repository query level, not just application logic.

```
-- repositories/chunk_repository.py (conceptual query)
SELECT chunk_id, memory_id, content, embedding <=> :query_vector AS distance
FROM memory_chunks
WHERE user_id = :user_id
ORDER BY distance ASC
LIMIT :top_k;
```

10.2 Metadata Filtering

Optional filters (source type, date range, tags) are applied alongside the vector search, either as a pre-filter (narrowing the candidate set before ranking) or a post-filter, depending on filter selectivity. `rag/retriever.py` decides which strategy to use based on estimated filter cardinality.

10.3 Ranking

Initial candidates are ordered by vector distance. A lightweight re-ranking step in `rag/ranker.py` can incorporate recency and engagement-signal weighting (e.g., favoring memories the user spent more time on) to break ties among semantically similar chunks — this is a tunable scoring function, not a separate ML model, for MVP.

10.4 Context Retrieval & Assembly

The top-K ranked chunks are fetched with their full text and source metadata, then assembled into a context block with clear delimiters and per-chunk source identifiers, so the LLM can attribute claims back to specific memories.

10.5 Prompt Assembly

```
# rag/prompt_builder.py (conceptual)
SYSTEM_PROMPT = """You answer only using the provided context.
Cite each claim using the [n] marker matching its source.
If the context does not contain the answer, say so."""

def build_prompt(query: str, chunks: list[RankedChunk]) -> list[dict]:
    context = "\n\n".join(
        f"[{i+1}] (source: {c.source_title}) {c.content}"
        for i, c in enumerate(chunks)
    )
    return [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"},
    ]
```

10.6 Citation Generation

`services/citation_service.py` maps the `[n]` markers in the LLM's response back to the originating memory records, producing a citation list (memory ID, title, source type, URL where applicable) returned alongside the answer text — never inferred from the LLM's own claims about its sources.

10.7 LLM Response

The assembled prompt is sent to GPT-4.1 (or GPT-5 where configured) via `rag/llm_client.py`. The MVP scope is strictly Semantic Search + RAG Chat + Source Citations — grounded, multi-document summarization beyond direct chat responses is explicitly deferred to V1.x per the PRD.

Scope reminder: per PRD v2.1, standalone "grounded summaries" as a distinct feature are **not** in MVP. The chat endpoint may produce a synthesized answer, but a dedicated summarization API/UI surface is post-MVP.

11. Background Jobs (Celery)

Celery, backed by Redis as both broker and result store, handles all asynchronous work: content processing, embedding generation, re-indexing, and scheduled maintenance.

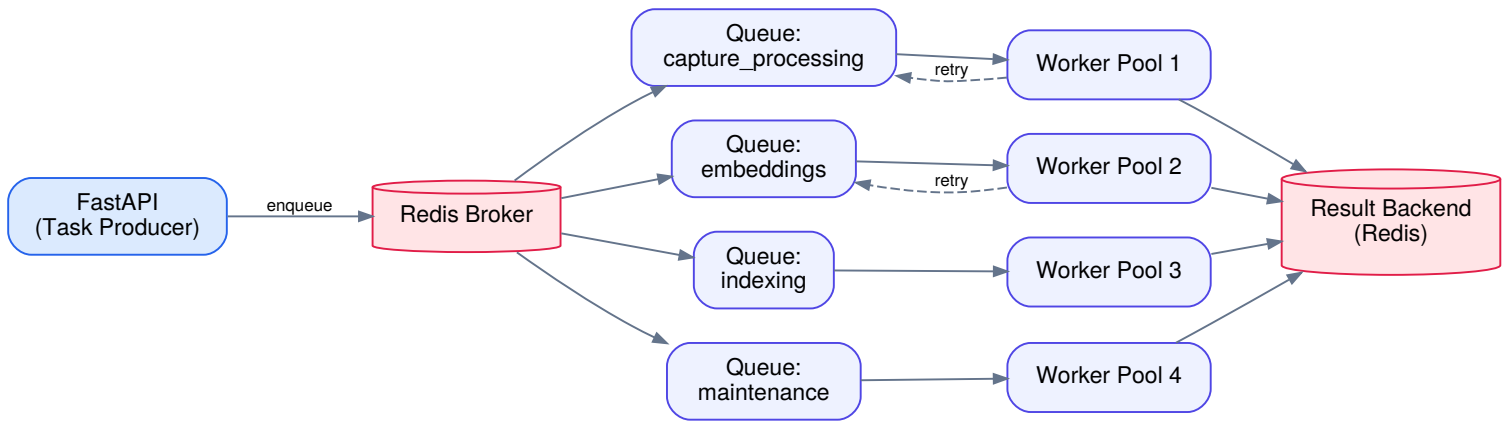


Figure 11.1 — Celery queue architecture and worker pools

11.1 Queue Design

Queue	Task Types	Priority	Concurrency Notes
capture_processing	Extraction, classification, cleaning	High	Scaled with capture volume; I/O-bound, higher concurrency per worker
embeddings	Batch embedding calls to OpenAI	High	Concurrency bounded by OpenAI rate limits, not CPU
indexing	Vector storage finalization, search index updates	Medium	Fast, DB-bound tasks
maintenance	Re-indexing, stale-vector cleanup, scheduled housekeeping	Low	Runs off-peak via Celery Beat schedules

11.2 Workers

Each queue is served by an independently scalable worker pool (separate Celery worker processes/containers), so a spike in capture volume does not starve embedding or maintenance work, and vice versa. Worker pools are stateless and can be scaled horizontally by adding container replicas.

11.3 Retries

Task Category	Retry Strategy	Max Attempts
External API calls (embeddings, LLM, YouTube, GitHub)	Exponential backoff with jitter	5
Database writes	Immediate retry, then backoff	3
Content extraction	Linear backoff	3

11.4 Task States

Every long-running job is tracked in a `jobs` table (via `JobRepository`), independent of Celery's own result backend, so job status survives Redis flushes and is queryable through the API for client polling.

State	Meaning
queued	Task accepted, not yet started
running	Task actively executing
retrying	Previous attempt failed, backoff in progress
completed	Task finished successfully
failed	All retries exhausted

11.5 Async Processing Pattern

```
# workers/capture_tasks.py
```

```
@celery_app.task(bind=True, max_retries=3, queue="capture_processing")
def process_capture(self, job_id: str):
    try:
        capture_service.run_pipeline(job_id)
    except TransientError as exc:
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)
```

12. Storage Architecture

Sentiora uses three storage systems, each responsible for a distinct kind of data. No storage system duplicates another's role — this separation keeps backup, access-control, and scaling decisions independent per system.

System	Stores	Access Pattern	Why This System
PostgreSQL	Users, memories, sources, jobs, consent settings, chunk metadata	Transactional reads/writes, relational joins, per-user filtering	ACID guarantees for consent and ownership data are non-negotiable
pgvector (inside PostgreSQL)	Chunk embedding vectors, linked to chunk metadata	Approximate nearest-neighbor similarity search, filtered by <code>user_id</code>	Keeps vector and relational data transactionally consistent — no separate vector DB to keep in sync
S3 / MinIO	Raw captured artifacts (PDFs, page snapshots) not needed for search itself	Write-once on capture, read on re-processing or export	Cheap, durable blob storage; keeps large payloads out of the relational database

12.1 Relationships Between Storage Systems

A single memory's data is distributed as follows: the **memory record** and its **chunk metadata** live in PostgreSQL; each chunk's **embedding vector** lives in the pgvector column on the same logical row (not a separate database); the **original raw artifact**, when retained, lives in S3 with its object key stored as a reference column in PostgreSQL. This means a full memory read never requires cross-system joins for search — only the archival raw content requires a separate S3 fetch, and only when explicitly requested (e.g., re-processing).

12.2 What Each System Does Not Store

- PostgreSQL does not store raw PDF/HTML blobs — those go to S3 to keep the database lean and backups fast.
- S3 does not store anything searchable directly — it is never queried at request time for search or chat.
- pgvector does not store content text redundantly beyond what's needed for retrieval display — full chunk text lives in the same row it's queried from, avoiding a second round-trip.

13. Error Handling & Logging

13.1 Global Exception Handling

A single FastAPI exception handler (`middleware/error_handler.py`) catches all unhandled exceptions and converts them into the standard error envelope (Section 6.6), ensuring the client never receives a raw stack trace or an inconsistent error shape.

```
@app.exception_handler(AppException)
async def app_exception_handler(request: Request, exc: AppException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": {"code": exc.code, "message": exc.message,
                           "request_id": request.state.request_id}},
    )
```

13.2 Error Taxonomy

Category	HTTP Status	Example Code	Cause
Validation Error	422	<code>VALIDATION_ERROR</code>	Request body fails Pydantic schema validation
Authentication Error	401	<code>UNAUTHORIZED</code>	Missing, expired, or invalid JWT
Authorization Error	403	<code>PERMISSION_DENIED</code>	Valid user, but action not permitted (e.g., consent not granted)
Not Found	404	<code>NOT_FOUND</code>	Resource does not exist or does not belong to the user
Database Error	500	<code>DB_ERROR</code>	Connection failure, constraint violation not caught earlier
AI/External Service Error	502 / 503	<code>AI_SERVICE_ERROR</code>	OpenAI API failure, timeout, or rate limit after retries exhausted
Rate Limited	429	<code>RATE_LIMITED</code>	Client exceeded request rate threshold

13.3 Validation Errors

Handled automatically by FastAPI/Pydantic; the standard handler reformats FastAPI's default validation error shape into Sentiora's error envelope so the format stays consistent across all error types.

13.4 Authentication Errors

Raised by the JWT middleware before a request reaches any route handler. No distinction is made in the response between "token expired" and "token invalid" to avoid leaking information useful for token forgery attempts; the distinction is available in server-side logs only.

13.5 Database & AI Errors

Both are caught at the service layer, wrapped into typed `AppException` subclasses (`DatabaseError` , `AIServiceError`), and re-raised so the global handler can map them to the correct status code. Raw driver/SDK exceptions never propagate to the API boundary.

13.6 Structured Logging

All logs are emitted as structured JSON (via `core/logging.py`) including `request_id` , `user_id` (when authenticated), route, status code, and latency. Celery tasks log with the originating `job_id` so a capture event can be traced end-to-end across the synchronous request and every downstream async task.

```
{"level": "error", "request_id": "req_9f8a3c", "user_id": "usr_123",
 "route": "/api/v1/memories", "error_code": "AI_SERVICE_ERROR",
 "message": "OpenAI embedding request timed out after 3 retries",
 "timestamp": "2026-07-29T10:15:03Z"}
```

13.7 Sample Error Response

```
HTTP/1.1 403 Forbidden
{
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Capture is not enabled for this source type.",
    "request_id": "req_9f8a3c"
  }
}
```

14. Security & Privacy

Concern	Implementation
JWT Security	Short-lived access tokens (15 min), rotating single-use refresh tokens, signing secret held in Secrets Manager (never in code or environment files committed to source control)
HTTPS	All traffic terminates TLS at the load balancer; the API never accepts plaintext HTTP in any environment beyond local development
Rate Limiting	Per-user and per-IP rate limits enforced at the API gateway/middleware layer, with stricter limits on auth endpoints to mitigate credential stuffing
Input Validation	Every request body validated by Pydantic schemas; no raw client input reaches a repository query unvalidated
SQL Injection Prevention	SQLAlchemy ORM with parameterized queries exclusively; no string-concatenated SQL anywhere in the codebase
XSS Prevention	Backend treats all captured content as untrusted text — never rendered as HTML server-side; the extension is responsible for safe rendering client-side
Secrets Management	Database credentials, JWT signing key, OpenAI API key, and S3 credentials loaded exclusively from AWS Secrets Manager / environment injection at deploy time
Permission Validation	Enforced server-side at capture time (Section 7.2), independent of and not trusting client-side consent UI state
Data Isolation	Every repository query scoped to the authenticated <code>user_id</code> ; no endpoint accepts a cross-user resource ID without ownership verification
Privacy-First Design	Never-capture policy enforced before persistence (Section 7.3); no password capture under any circumstance; raw sensitive content is never logged

14.1 Defense in Depth for Data Isolation

Data isolation is enforced at two independent layers so a bug in one does not expose another user's data:

- Service layer** — every service method receives an authenticated `user_id` from the route's dependency-injected context, never from the request body.
- Repository layer** — every query includes a `WHERE user_id = :user_id` clause as a non-optional parameter of the repository method signature, not an afterthought filter.

14.2 Secrets Handling Rules

- No secret value is ever committed to source control, including in `.env.example` files (placeholders only).
- Local development uses a `.env` file excluded via `.gitignore` ; production uses AWS Secrets Manager injected as environment variables at container start.
- Secrets are never logged, even at debug level — logging middleware redacts known secret-shaped fields defensively.

15. Deployment

15.1 Development Environment

Local development runs entirely via Docker Compose: FastAPI app, PostgreSQL with pgvector, Redis, a Celery worker, and MinIO as an S3-compatible substitute — no AWS account required to develop locally.

```
# docker-compose.yml (conceptual)
services:
  api:
    build: .
    ports: ["8000:8000"]
    depends_on: [postgres, redis, minio]
  worker:
    build: .
    command: celery -A workers.celery_app worker --loglevel=info
    depends_on: [postgres, redis]
  postgres:
    image: ankane/pgvector
    environment: [POSTGRES_DB=sentiora]
  redis:
    image: redis:7
  minio:
    image: minio/minio
    command: server /data
```

15.2 Production Deployment (AWS)

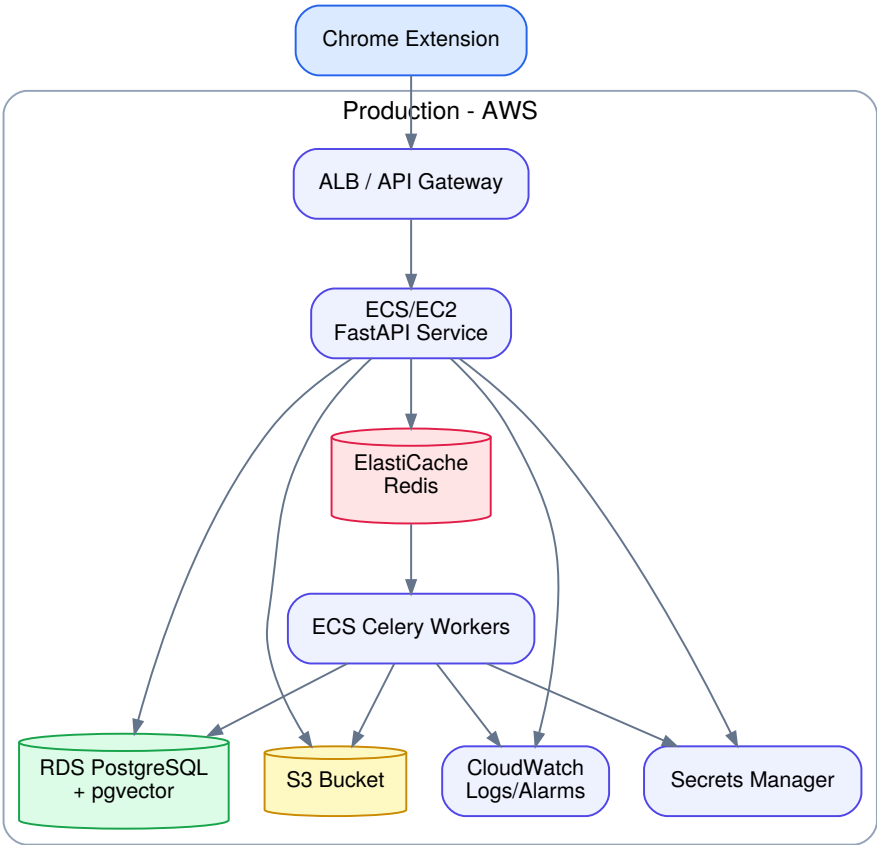


Figure 15.1 — Production deployment topology on AWS

Component	AWS Service	Notes
API compute	ECS (Fargate) or EC2 behind an ALB	Stateless containers, horizontally scaled on CPU/request metrics
Celery workers	ECS (Fargate) — separate service per queue priority	Scaled independently from the API service
Database	RDS PostgreSQL with pgvector extension enabled	Automated backups, single primary for MVP (no read replicas yet)
Cache/Broker	ElastiCache Redis	Used as both Celery broker and application cache
Object Storage	S3	Versioning enabled for accidental-deletion recovery
Secrets	AWS Secrets Manager	Injected as environment variables at container start
Logs/Monitoring	CloudWatch Logs + Alarms	Structured JSON logs shipped from all containers

15.3 Environment Variables

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string
<code>REDIS_URL</code>	Redis connection string (broker + cache)
<code>JWT_SECRET_KEY</code>	Signing key for access/refresh tokens
<code>OPENAI_API_KEY</code>	Embedding and LLM API access
<code>S3_BUCKET_NAME</code> / <code>S3_ENDPOINT_URL</code>	Object storage target (MinIO endpoint in dev, S3 in prod)
<code>ENVIRONMENT</code>	Toggles dev/staging/prod behavior (e.g., debug logging, CORS origins)

16. Cross-Cutting Concerns

Concern	What It Is	Why It Exists	How It Works
Middleware	Request ID, CORS, auth, error handling, logging (Section 6.7)	Consistent behavior across every route without duplicating logic	Registered once in <code>main.py</code> , applied to all requests in a fixed order
Configuration Management	Centralized <code>core/config.py</code> using Pydantic Settings	Avoids scattered <code>os.environ</code> calls and untyped config	Loaded once at startup, validated, injected via dependency where needed
Environment Variables	All environment-specific values (Section 15.3)	Same codebase runs in dev/staging/prod without code changes	Read exclusively through <code>core/config.py</code> , never accessed ad hoc
Dependency Injection	FastAPI's <code>Depends()</code> for DB sessions, current user, services	Testability and explicit dependencies over hidden globals	Providers defined in <code>api/deps.py</code>
Request IDs	UUID generated per request, propagated into Celery task args	End-to-end tracing across sync and async boundaries	Assigned in the first middleware, included in every log line and error response
Audit Logging	Append-only log of consent decisions and never-capture discards	Accountability for privacy-sensitive decisions without storing the sensitive content itself	Written by <code>PermissionService</code> , stores only metadata (timestamp, decision, source type)
Health Checks	<code>/health</code> endpoint checking DB, Redis, and S3 connectivity	Required for ALB target group health and container orchestration	Lightweight, unauthenticated, no business logic invoked

17. Future Module Extension

Sentiora Footprint and Sentiora Shield are explicitly post-MVP (per PRD v2.1). This section defines only the backend **extension points** the MVP architecture already accommodates — it is not an implementation design for either module, and no Footprint/Shield tables, services, or endpoints are built in MVP.

17.1 Sentiora Footprint Post-MVP

Footprint maps accounts, services, auth relationships, and identity graph — explicitly separate from Memory's activity content.

Extension Point	How the MVP Architecture Accommodates It
New services	A new <code>services/footprint_service.py</code> can be added without touching Memory/Intelligence services — the layered architecture (Section 3) does not require cross-module coupling
New APIs	New routes under <code>/api/v1/footprint/</code> , following the same route → service → repository pattern already established
New tables	New tables (e.g., <code>identity_accounts</code> , <code>service_permissions</code>) added via Alembic migration, referencing <code>users.id</code> ; no changes required to existing Memory tables
New workers	A new Celery queue (e.g., <code>footprint_scan</code>) can be added to the existing Celery app without affecting Memory/Intelligence queues

17.2 Sentiora Shield Post-MVP

Shield consumes Footprint and Memory signals to flag risks; it does not independently collect data.

Extension Point	How the MVP Architecture Accommodates It
Risk analysis service	Reads from existing Memory and (future) Footprint repositories as a consumer, not a new data source — respects the same per-user data isolation rules (Section 14.1)
Privacy analysis	Built as a scoring layer over existing consent/audit data (Section 16), no new capture path required
Security scoring	Computed asynchronously via Celery, following the existing job-tracking pattern (Section 11.4)
Alert pipeline	Can reuse the existing job/notification infrastructure; exact delivery mechanism (in-app, email) is a Shield-phase decision, not defined here

Shield's exact architecture is intentionally left undefined at this stage, consistent with the PRD's Decisions Register, to avoid locking in unfinalized product decisions. It should be scoped as its own technical specification once Shield's product requirements are finalized.

18. Development Roadmap

Recommended implementation order for the 4-person engineering team, sequenced so each phase produces a testable increment and later phases only depend on already-completed ones.

Phase	Focus	Key Deliverables	Depends On
Phase 1	Authentication	User model, registration, login, JWT issuance/refresh, auth middleware	—
Phase 2	Database	Full schema migration (per Database Schema doc), repository layer scaffolding, pgvector extension setup	Phase 1
Phase 3	Capture APIs	Capture endpoint, permission validation service, consent settings API	Phases 1–2
Phase 4	Processing Pipeline	Content extractors (web, PDF first), normalization, chunking, Celery task wiring	Phase 3
Phase 5	Embeddings	Embedding client, batching, vector storage, embedding lifecycle tracking	Phase 4
Phase 6	Semantic Search	Vector search endpoint, metadata filtering, ranking	Phase 5
Phase 7	AI Chat	RAG retrieval + prompt assembly + LLM integration + citation generation	Phase 6

Remaining content extractors (YouTube, GitHub, manual notes, AI conversations) can be parallelized across the team from Phase 4 onward once the base extractor interface (Section 8.1) is in place; they do not block Phase 5–7 progress on the web/PDF path.